mRPL++ : Smarter-HOP for optimizing mobility in RPL

Radhesh Anand M C and Mohit P. Tahiliani
Wireless Information Networking Group (WiNG)
NITK, Surathkal, Mangalore, India, 575025
Email: radheshanand@gmail.com, tahiliani@nitk.ac.in

***Abstract*—Routing Protocol for Low power and lossy networks (RPL) is a proactive algorithm for Low-power and Lossy Networks (LLNs). Recent growth in Internet of Things (IoT) applications has made proactive handling of mobility apparent. While RPL does not tackle mobility, mRPL addresses it by adopting a proactive Hand-Off strategy. Since this strategy follows a greedy approach to select the next hop, a best route may not be always chosen. This paper proposes mRPL++, an extension of mRPL, to ensure best route selection in mobile scenarios. We have implemented mRPL++ in the Contiki 6LoWPAN/RPL stack and validated it through extensive simulations.**

***Keywords:* Internet of Things, Routing, RPL, Mobility, mRPL, Hand-off, Performance evaluation, Simulation**

1. Introduction

The term "Internet of Things" was coined by British entrepreneur Kevin Ashton in 1999. IoT, also known as next generation Internet, depicts a world populated by an endless number of smart devices that are able to sense, process, react to the environment, cooperate and intercommunicate via the Internet [1]. The research and developments in the field of IoT in recent past have progressed the state of the art, opening up the whole new world of possibilities and new dimensions. Low-power and Lossy Networks (LLN) [6] consist of constrained nodes, interconnected by lossy links, low data rates, low packet delivery ratio, and have different traffic patterns like point-to-point, point-to-multipoint or multipoint-to-point. These characteristics of LLN offer unique challenges to formulate solutions for routing. IETF designed 6LoWPAN, an adaptation layer that supports routing in link layer (mesh-under) and network layer (route-over) [2]. Mesh-under is most suited for single-hop networks where as route-over for multi-hop networks.

Routing protocols are usually of two types [10]: proactive and reactive. In case of proactive protocols, routes are computed and maintained, independent of the data transfer. If the node is currently unreachable, these protocols fail to provide the routes. The other drawbacks in proactive protocols are: the memory used to store the routing information and the overhead of maintaining routes that may never be used. In case of reactive protocols, route to a particular node to which the data is to be delivered is discovered on-demand. Major drawback of reactive protocols is the delay involved in gathering the route information.

Taking into consideration of the drawbacks of both the types of protocols, RPL follows a reactive approach, with necessary control measures to overcome them. RPL implements an algorithm called trickle, and also uses Neighbor Discovery (ND) algorithm of 6LoWPAN to handle route failures. These algorithms are triggered only when the failures are detected. This approach does not suit well when nodes are mobile because mobility leads to frequent route failures and subsequently increases the packet drops and latency. Recent applications of IoT require timeliness and reliability guarantees for transmitting critical messages and cannot tolerate packet losses or high latency. Moreover, for many IoT applications such as health-care monitoring, human-robots and smart grid, mobility support is one of the core requirements. As a result, the need for proactively handling mobility has become apparent.

A lot of attempts have been made in the recent past to make RPL mobile compliant. The most promising approach is the one that is adopted by mRPL [1]. mRPL adopts a proactive HO approach to minimize packet losses and latency that arise due to mobility. HO approach has its roots in cellular and WLAN networks, and has proven to be robust so far. The HO approach used in mRPL is called "dubbed smart-HOP mechanism" and has been tested under homogeneous networks with fixed node infrastructure that support source node mobility. While mRPL successfully handles node mobility, there are a few possibilities when it may end up selecting a sub-optimal path from source to destination. This is because the "dubbed smart-HOP" is a greedy mechanism that chooses the next best "local hop" based on average RSSI (ARSSI). The next best "local hop" may not be a part of an optimal path.

In this paper, we propose a smarter-HOP mechanism that not only considers ARSSI values to choose the next best hop, but also incorporates the OF into mRPL. Adding OF to smart-HOP mechanism makes it smarter. We call the enhanced mRPL as mRPL++.

Why make smart-HOP smarter? HO is well studied in case of cellular and WLAN networks, where only the last hop uses wireless medium and the backbone network usually uses wired medium. Hence, a greedy approach suffices in such scenarios. Conversely, an IoT network mostly uses only wireless medium, due to which a greedy approach during HO may lead in selection of sub-optimal path. Also, RPL proactively builds topologies that are basically Directed Acyclic Graphs (DAGs) directed towards the root node (usually a RPL border router, that connects these devices to the external world). In case of RPL, DAGs are constructed based on the OF and the same OF is used to reconstruct the DAGs in case of route failures. Whereas, in case of mRPL only the ARSSI value is considered for selecting the next best hop, and reconstructs

978-1-5090-0931-2/16/$31.00 ©2016 IEEE

the DAG accordingly. The resulting DAG, however may not be OF compliant since the HO is done without considering OF.

Organization of the paper: Section 2 provides the necessary background about RPL and mRPL. Section 3 describes the general design of the proposed smarter-HOP mechanism. The simulation environment followed by the results are presented in Section 4. Finally, we conclude the paper and outline the most relevant findings in Section 5.

2. Background

2.1. RPL

A lossy link is not only characterized by a high Bit Error Rate (BER), but also the inaccessibility time which strongly impacts the routing protocol behavior. Hence, RPL is designed to be highly adaptive to network conditions and provides alternate routes whenever default routes are inaccessible. RPL's topological concept, Destination Oriented Directed Acyclic Graph (DODAG), is derived from the concept of DAG which defines a tree structure. DODAG is a logical routing tree built over a physical network depending on a specific OF. Further, DODAG is periodically reconstructed to enable global reoptimization using the trickle timer approach. Thus, RPL can be used by different applications of LLN by choosing an appropriate OF.

Like Ad hoc On demand Distance Vector (AODV) protocol, RPL mainly operates in two phases: route discovery and route maintenance. New routes are formed during the route discovery phase; and once formed, they are maintained during the route maintenance phase. A list of control messages used during route discovery and route maintenance is summarized in Table 1. Moreover, different identifiers used during these phases are:

- RPL Instance ID: different RPL instances can be created for different OFs. Each one is identified by a unique RPL instance ID.
- DODAGID: Since there can be multiple DODAGs to account for each RPL instances, every DODAG is identified by a DODAGID. The combination of RPL Instance ID and DODAGID uniquely identifies a single DODAG.
- DODAGVersionNumber: A particular DODAG can be re-constructed several times to account for route failures. Each time the DODAG is re-constructed, a new DODAGVersionNumber is assigned to it.
- Rank: Each node in a DODAG has a rank, which is equal to the number of hops between that node and the root node. These ranks are exchanged between nodes in order to avoid loops.

TABLE 1. RPL CONTROL MESSAGES

Control messages in RPL	Use	Direction
DIO:DODAG Information Object	Upward path creation	From Source to Destination only
DAO:DODAG Advertisement Object	Downward path creation	From Destination to Source only
DIS:DODAG Information Solicitation	Link or node failure	Neighboring nodes
CC:Consistency Check	Link and node consistency check	Timely broadcast message

Detecting topology changes in RPL

RPL incorporates the following two approaches to detect topology changes in the network:

(a) *Trickle timer approach*: Traditional algorithms in LLNs broadcast control messages at fixed intervals to detect changes in the topology. Small intervals lead to significant increase in the control message overhead, which further leads to high probability of collision and subsequently, loss of packets. Large intervals, on the other side, induce very little control message overhead, but cannot cope up with frequent route failures. A variable timer, instead of a fixed one, is required to carefully balance the trade off between control message overhead and timely detection of topology changes. Trickle timer used by RPL does something similar. The idea is to gradually converge the periodic timer value to its maximum value depending on the stability of the network topology and reset to its minimum value when any inconsistency is detected. Value of trickle timer is bound by the interval [I_{MIN}, I_{MAX}], where $2^{I_{MIN}}$ is the minimum timer value defined in milliseconds and maximum timer value is calculated using a doubling factor $I_{doubling}$ as: $I_{MAX} = I_{MIN}$ x $2^{I_{doubling}}$.

(b) *IPv6 neighbor discovery approach*: Another approach adopted by RPL to detect topology changes is the usage of Neighbor Discovery (ND) protocol of 6LoWPAN that has been optimized by IETF for LLNs [9]. ND protocol supports four ICMPv6 messages: (i) Neighbor Solicitation (NS): used to determine the reachability of the neighbor, (ii) Neighbor Advertisement (NA): used as a reply message to NS and also used periodically to announce link changes, (iii) Router Solicitation (RS): used to request for information from the router, and (iv) Router Advertisement (RA): sent by the router periodically advertising its presence and also in response to RS. RA is also used to indicate regarding its failure or restart, by advertising the value 0 to all the nodes it serves.

It can be noted that both the above mentioned approaches are reactive and hence, are not suitable for mobile networks. Mobility leads to frequent topology changes that should be proactively handled. One variant of RPL that effectively handles mobility is explained in the next section.

2.2. mobility compliant RPL (mRPL)

Unlike RPL, mRPL proactively handles mobility by using smart-HOP algorithm. smart-HOP is a proactive algorithm that timely initiates a HO process and attempts to minimize the packet losses due to frequent handovers. Further, mRPL uses four timers to perceive and resolve link inaccessibility problem, which are explained later.

smart-HOP algorithm

smart-HOP algorithm is initiated during the route maintenance phase of RPL and its functionality is divided into following two stages:

(a) *Data transmission stage:* In this stage, the quality of a link between the parent and Mobile Node (MN) is measured in terms of ARSSI. The procedure to calculate ARSSI is as follows: (i) the parent node, on receiving *k* data packets in a window from the MN, calculates the ARSSI and sends it to the MN (the average of RSSI is taken to account for highly variable nature of LLN links). If the ARSSI value is in the transitional region (T_I < ARSSI < T_H) defined by Hysteresis Margin (HM) [4], MN initiates the discovery stage, else it continues with the data transmission stage. If T_I is set very high, MN may initiate many unnecessary HOs. On the other hand, if T_I is set very low, MN may continue to use unreliable links. Similarly, if the value of HM is low, unnecessary HOs could occur between two potential parents (this is commonly known as the ping-pong effect [12]). On the contrary, if the value of HM is high, the HO process may take too long resulting in packet losses and increase in latency.

(b) *Discovery stage*: As mentioned above, if the quality of the link degrades below T_I threshold, MN initiates the discovery stage. During this stage, MN broadcasts DIS messages to all its neighboring nodes and receives a unicast DIO message along with an ARSSI value from each neighbor in a non-collision manner. MN stores the details of a neighbor only if the received ARSSI value is greater than T_I, others are discarded. Finally, the MN chooses a neighbor that has the highest ARSSI value as a parent, connects to it and resumes data transmission.

Timers

mRPL uses the following four timers to ensure smooth functioning of the protocol when nodes are mobile:

- *Connectivity timer (T_C)*: The MN constantly monitors the link and the incoming packets from the serving parent. T_C elapses if the parent is silent, triggering the discovery stage. Upon any packet reception from the serving parent, the timer is reset. The timer value is set according to the maximum Trickle interval (I_{MAX}). It is basically used to increase the responsiveness of the routing protocol.
- *Mobility detection timer (T_{MD})*: During the data transmission stage, DIS is periodically sent requesting a unicast DIO message from the serving parent in order to read the ARSSI value. When the ARSSI value drops below T_I due to mobility or obstacle between the node and its serving parent, the MN initiates discovery stage. The timer value is set according to the data generation rate.
- *HO timer (T_{HO})*: It is used to reduce the HO delay. It manages the periodicity of broadcasting of DIS to the neighboring parents during the discovery phase.
- *Reply timer (T_R)*: Its main functionality is to avoid collision of DIS and unicast DIO messages with data and is calculated as: (ws - C) x T_{DIS}, where C represents the DIS counter within each window size (ws) and T_{DIS} represents the DIS interval. It adaptively changes upon receiving new packets.

While mRPL handles mobility quite efficiently by using the greedy approach of selecting the next parent node based on ARSSI, there is a possibility of a non-optimal route being selected. To tackle this problem, we propose a smarter-HOP mechanism, described in the next section.

3. Smarter-HOP

Smarter-HOP intends to better the performance of smart-HOP in compliance with the requirements of RPL. The heart of RPL, as mentioned before, is the OF. OF can be of two types : (i) minimize function - lesser the better, e.g.: expected transmission count (ETX) or (ii) maximize function - higher the better, e.g.: rank. Figure 1 illustrates a simple condition of hand-over, where an MN moves out of range of AP_k and enters in an overlapping range of two APs: AP_n and AP_m, with OF values X and Y respectively (assumptions made are: OF is of type maximize function and X > Y).

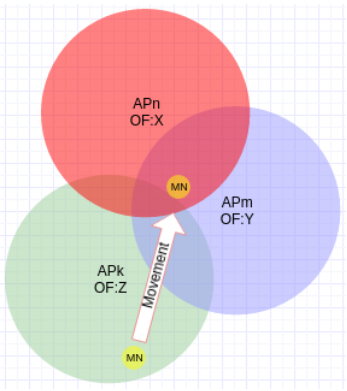


Figure 1. Hand-over scenario

From Figure 1, it is evident that ARSSI AP_m > ARSSI AP_n . Thus, if the MN was to choose the AP based on OF as in RPL, it would have chosen AP_n. On the contrary, it would have chosen AP_m based on ARSSI value as in mRPL. However, in case of our proposed mRPL++ with smarter-HOP mechanism, the AP would be chosen based on both OF and ARSSI value, making efforts towards satisfying the criteria of selecting the best possible parent with minimized probability of defying the thumb rule of routing (i.e., OF).

Introducing a new variable for smarter-HOP mechanism would increase the manual intervention required to fine tune the variables for every different type of network. To avoid this, we consider the product of the ratio of average OF value and OF value of the possible parent node, and the ARSSI value during decision making phase. DIO messages sent by each

begin: if *received a unicast DIO from all neighbor nodes* **then**
  calculate the avgOF of all neighbor nodes;
  for *i in neighbor nodes* **do**
    if *OF type is minimize* **then**
      $bestNeighborValue_i$ = avgOF/OF_i * $ARSSI_i$;
    else
      $bestNeighborValue_i$ = OF_i/avgOF * $ARSSI_i$;
    end
    if *bestNeighborValue* $< bestNeighborValue_i$ **then**
      bestNeighborValue = $bestNeighborValue_i$;
      Save the bestNeighbor's address as the next parent node;
    end
  end
  Send unicast DAO requesting to connect to the bestNeighbor;
else
  Continue Discovery phase;
end

Algorithm 1: Parent selection

parent contains the OF values, hence we can easily derive the value without much overhead. The ratio is self maintained with respect to the type of OF as illustrated in Algorithm 1.

4. Simulation

In order to compare the performance of mRPL with mRPL++, we carried out simulations using COOJA, a network simulator embedded within the CONTIKI operating system [13]. We simulate mRPL (smart-HOP) and the proposed algorithm (mRPL++) considering different settings and topologies as discussed in the latter part of the section. ETX is one of the most used OF in the field of IoT, globally. The authors of [5] claim that ETX gives better performance over OF0 [7], and thereby, we have used ETX for our simulations of mRPL++.

4.1. Setup

The major factors that affect the performance of mRPL are the timer values. According to the authors of mRPL [1], the timer values are dependent on the I_{min} and $I_{doubling}$ values of the trickle algorithm. The four different value combinations used for comparison are shown in Table 2.

TABLE 2. DESCRIPTION OF THE SCENARIOS

Scenarios	I_{min}	$I_{doubling}$
8_1	8	1
10_2	10	2
12_1	12	1
12_8	12	8

The following network metrics have been considered for the evaluation of mRPL and mRPL++:

- Packet delivery ratio (PDR) - number of successfully received packets at all access points over the total number of packets sent from the MN.
- Overhead with respect to control messages - calculated as total number of control messages over total number of control and data messages.
- Total DAO packets - indicates the responsiveness and number of HOs in mobility enabled network.
- Average latency - represents the average time required for the packet to reach the destination.

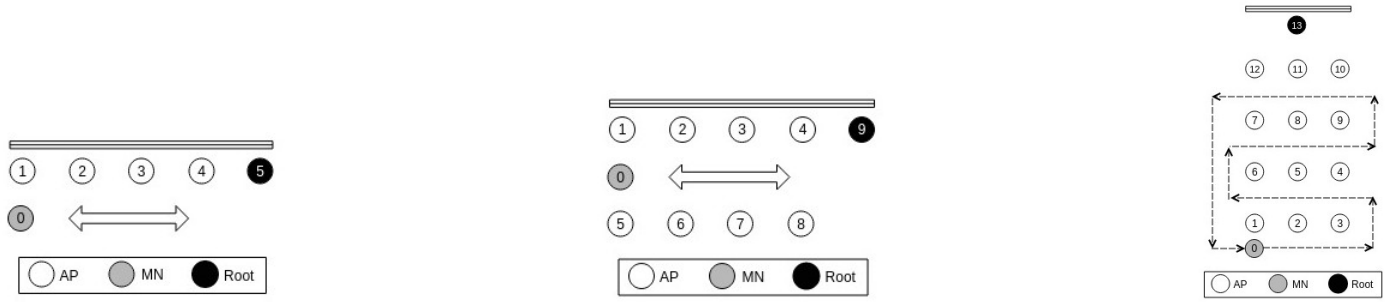
Assuming a health-care center scenario, three different topologies are defined and used, to compare the performance of two protocols: (i) Topology I: four access-points as shown in the Figure 2a (corridor), (ii) Topology II: eight access-points as shown in the Figure 2b (corridors across departments), and (iii) Topology III: twelve access-points as shown in the Figure 2c (across different floors and departments), with one MN and a server. The speed of the MN is kept constant and data generation rate is fixed at 30 packets/second as suggested by the authors of mRPL [1].

4.2. Evaluation result

The comparison results obtained for each scenario are represented in a graphical format, per topology. Figures 3, 4 and 5 show that mRPL++ performs better in most of the scenarios by improving the packet delivery ratio, lowering control overhead thereby improving the lifetime of the network, and also lowers the average latency in the network significantly. The number DAO messages, average latency and overhead are reduced in most of the scenarios hinting towards less number of HOs. The reasons for better performance of mRPL++ are explained below.

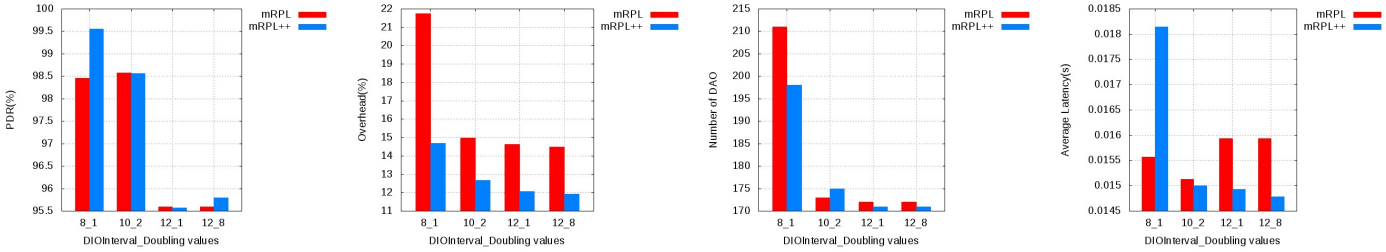
Topology I depicts a normal scenario where a MN moves either towards or against the root node. mRPL and mRPL++ are not left with much choice in this topology as the APs are arranged sequentially on one side only. It can be observed from the graphs that the performance of both protocol varies marginally and in some cases, mRPL++ ends up with more latency and more number of DAOs. This is primarily because when a MN moves against the root node using mRPL++, it does not find the next parent with a better OF and thus, delays the HO or invokes a global DAG maintenance. Topology II gives many options to mRPL++ (i.e., APs to choose) while a MN moves towards or against the root node. Due to availability of many options, HO process in mRPL++ does not get delayed and hence, we observe that it consistently outperforms mRPL in all parameters. Topology III is a combination of I and II and involves much more mobility patterns than the previous both. In some parameters, mRPL++ does not perform well mainly due to the presence of some areas where only a single AP is available to choose as a parent (e.g.: when a MN is moving across different floors). In these areas, the performance of mRPL++ falls back to that obtained during topology I because of delayed HO or invocation of global DAG maintenance.

Our observations based on the simulation of these three topologies indicate that mRPL++ is a better protocol for



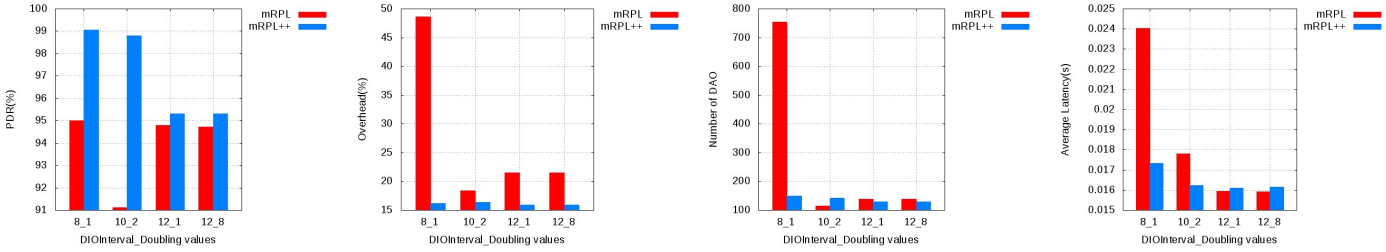
(a) Topology I (b) Topology II (c) Topology III

Figure 2. Different topologies used



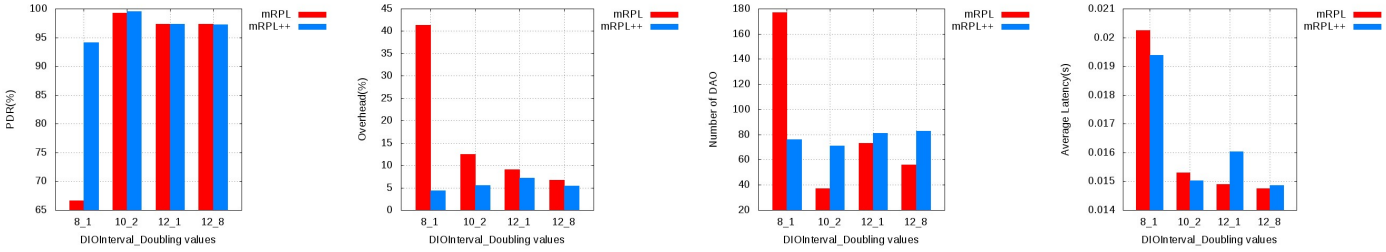
(a) Packet delivery ratio (b) Control message overhead (c) Total number of DAOs (d) Average latency

Figure 3. Results of Topology I



(a) Packet delivery ratio (b) Control message overhead (c) Total number of DAOs (d) Average latency

Figure 4. Results of Topology II



(a) Packet delivery ratio (b) Control message overhead (c) Total number of DAOs (d) Average latency

Figure 5. Results of Topology III

applications that have densely deployed APs. The lack of availability of sufficient APs hurts the performance of mRPL++. Nevertheless, mRPL++ can be optimized to work well in such scenarios by carefully tuning the equation used in algorithm 1.

5. Conclusion and future directions

The performance benefits obtained in case of mRPL++ second the thought of considering OF for the decision making of HO to improve the performance of traditional RPL. Native

distance vector algorithms also consider OF (e.g.: weight) for the decision making of HO to improve the performance of the network. However, there's a need for extensive testing of the smarter-HOP mechanism across different topologies and mobility models. Moreover, fine tuning of the previously used variables is highly desired to perform actual testing in testbed scenarios.

Acknowledgments

The authors would like to thank Hossein Fotouhi and Daniel Moreira for helping with the implementation of mRPL in COOJA, and Shivkumar, Christy Ouseph, Swapna Chalavindala, Hitaswi Nasari, Bibek Luitel, Pranav Vankar and Dharmendra Mishra, for their valuable support and feedback.

References

[1] Fotouhi, Hossein, Daniel Moreira, and Mrio Alves, *mRPL: Boosting mobility in the Internet of Things*, Ad Hoc Networks 26, 2015.

[2] A.H. Chowdhury, M. Ikram, H.-S. Cha, H. Redwan, S.M.S. Shams, K.-H. Kim, S.-W. Yoo, *Route-over vs mesh-under routing in 6loWPAN*, ACM, 2009.

[3] H. Fotouhi, M. Zuniga, M. Alves, A. Koubaa and P. Marrn, *Smart-hop: a reliable handoff mechanism for mobile wireless sensor networks*, EWSN, 2012.

[4] Zonoozi, Mahmood, and Prem Dassanayake, *Handover delay and hysteresis margin in microcells and macrocells.*, PIMRC'97, IEEE, 1997.

[5] Sharma, Rahul, and T. Jayavignesh, *Quantitative Analysis and Evaluation of RPL with Various Objective Functions for 6LoWPAN*, Indian Journal of Science and Technology, 2015.

[6] T. Winter, P. Thubert, A. Brandt, J. Hui *Rpl: IPv6 Routing Protocol for Low-Power and Lossy Networks*, Internet Proposed Standard RFC 6550, 2012.

[7] P. Thubert, *Objective Function Zero for the Routing Protocol for Low-Power and Lossy Networks (RPL)*, Internet Proposed Standard RFC 6552, 2012.

[8] J. Hui and JP. Vasseur, *The Routing Protocol for Low-Power and Lossy Networks (RPL) Option for Carrying RPL Information in Data-Plane Datagrams*, Internet Proposed Standard RFC 6552, 2012.

[9] G. Montenegro and N. Kushalnagar, *Neighbor Discovery Optimization for IPv6 over Low-Power Wireless Personal Area Networks (6LoWPANs)*, Internet Proposed Standard RFC 6775, 2012.

[10] Tripathi, Joydeep, Jaudelice C. De Oliveira, and J. P. Vasseur, *Proactive versus reactive routing in low power and lossy networks: Performance analysis and scalability improvements*, Ad Hoc Networks 23, 2014.

[11] Zekri, Mariem, Badii Jouaber, and Djamal Zeghlache. *Context aware vertical handover decision making in heterogeneous wireless networks*, 2010 IEEE 35th Conference on. IEEE, 2010.

[12] Kinan Ghanem, Haysam Alradwan, Adnan Motermawy, Ahmad Ahmad, *Reducing Ping-Pong Handover Effects In Intra E-UTRA Networks*, 8th IEEE, IET International Symposium on Communication Systems, 2012

[13] *Contiki-OS*[Online]. Available:www.contiki-os.org